

MODULE 3 EXPERIMENT REPORT

Python BDD RESTful Automations

Integrated experiment covering Python basics, REST API automation, HTTP response handling, database-driven testing, and Behave BDD framework implementation.

1. Module Name and Experiment Title

Module 3.0: Python BDD RESTful Automations

Experiment Title: End-to-End REST API Automation Using Python Requests, MySQL, Behave BDD, Hooks, Tags, and Allure Reporting

2. Problem Statement

Manual API testing becomes repetitive and difficult to maintain when multiple endpoints, payloads, authentication methods, response validations, database records, and execution environments are involved. The objective of this experiment is to design a reusable Python-based REST API automation framework that validates HTTP methods, response content, status codes, headers, authentication, sessions, cookies, database-driven data, and BDD scenarios.

3. Objectives

- Understand REST and SOAP and identify the characteristics of RESTful services.
- Set up Python, the requests library, and Behave for API automation.
- Automate GET, POST, PUT, PATCH, and DELETE requests.
- Validate status codes, headers, JSON payloads, text content, binary content, and error responses.
- Use configuration objects, reusable payloads, authentication, sessions, and cookies.
- Connect Python automation with MySQL for data-driven API testing.
- Create Behave feature files, step definitions, hooks, tags, and scenario outlines.
- Generate Allure reports for API BDD execution.

4. Tools, Software, and Concepts Used

- Python 3.x
- requests library
- Behave BDD framework
- Allure reporting framework
- MySQL and Python database connector
- JSON, HTTP methods, headers, status codes, authentication, cookies, and sessions
- Configuration files/objects and reusable test data
- IDE or terminal such as VS Code, PyCharm, or Command Prompt

5. Source Code and Implementation Steps

5.1 Python Basics, REST and SOAP

REST is an architectural style that uses resources and standard HTTP methods. REST APIs commonly exchange JSON and are generally stateless. SOAP is a protocol that commonly uses XML envelopes and formal messaging standards. In this experiment, Python and the requests library are used for REST API automation.

```
import requests

base_url = "https://example.com/api"
response = requests.get(f"{base_url}/users/1")
print(response.status_code)
print(response.json())
```

5.2 Python and Behave Setup

```
python -m venv venv
venv\Scripts\activate
pip install requests behave allure-behave mysql-connector-python
```

A typical project structure is:

```
api_automation/
├── features/
│   ├── environment.py
│   ├── users.feature
│   └── steps/
│       └── user_steps.py
├── utilities/
│   ├── config.py
│   └── api_client.py
├── test_data/
│   └── payloads.json
└── requirements.txt
```

5.3 GET Request and JSON Response

```
import requests

response = requests.get("https://example.com/api/users/1")
assert response.status_code == 200
data = response.json()
print(data)
```

The response object provides `status_code`, `headers`, `text`, `content`, `json()`, `cookies`, `elapsed`, and other properties used for validation.

5.4 Response Status Codes and Headers

```
assert response.status_code == 200
assert "content-type" in response.headers
assert "application/json" in response.headers["content-type"]
```

5.5 POST Request with Payload and Headers

```
payload = {
    "name": "Ahana",
    "role": "QA Engineer"
}
headers = {"Content-Type": "application/json"}

response = requests.post(
    "https://example.com/api/users",
    json=payload,
    headers=headers
)
assert response.status_code in [200, 201]
```

5.6 Configuration, Reusable Data, Authentication, Sessions and Cookies

```
import requests

session = requests.Session()
session.headers.update({"Accept": "application/json"})
session.headers.update({"Authorization": "Bearer <token>"})

response = session.get("https://example.com/api/profile")
print(response.cookies)
```

Global configuration can be stored in a Python configuration object or a separate configuration file. Payloads can be read from JSON, CSV, or database records so that test logic remains reusable. Session management reduces repeated setup and allows cookies and common headers to be retained across requests.

5.7 MySQL Data-Driven API Testing

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="api_test"
)

cursor = connection.cursor(dictionary=True)
cursor.execute("SELECT id, name, role FROM users")
rows = cursor.fetchall()

for row in rows:
    print(row)

cursor.close()
connection.close()
```

The database can provide input data for request payloads and expected values. Connections, cursors, and resources should be closed after execution, preferably through structured exception handling or context management.

5.8 PUT, PATCH and DELETE

```
put_response = requests.put(url, json=complete_payload)
patch_response = requests.patch(url, json={"role": "Tester"})
delete_response = requests.delete(url)
```

```
assert put_response.status_code in [200, 204]
assert patch_response.status_code in [200, 204]
assert delete_response.status_code in [200, 202, 204]
```

PUT generally replaces the complete resource representation, whereas PATCH is intended for partial modification. DELETE requests the removal of a resource. Exact status codes depend on the API contract.

5.9 Advanced Response Handling and Exceptions

```
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    text_data = response.text
    binary_data = response.content
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.RequestException as error:
    print("Request failed:", error)
```

Large responses may be processed incrementally with `stream=True`. Text data is accessed through `response.text`, binary data through `response.content`, and structured JSON through `response.json()`. Error handling should distinguish HTTP errors, timeouts, connection failures, and invalid JSON.

5.10 Behave BDD Feature File

Feature: User API validation

```
@smoke @api
Scenario: Retrieve a user successfully
    Given the API endpoint is available
    When I send a GET request for user "1"
    Then the response status code should be 200
    And the response should contain the user id
```

5.11 Step Definitions

```
from behave import given, when, then
import requests

@given("the API endpoint is available")
def step_endpoint(context):
    context.base_url = "https://example.com/api"

@when('I send a GET request for user "{user_id}"')
def step_get_user(context, user_id):
    context.response = requests.get(
        f"{context.base_url}/users/{user_id}"
    )

@then("the response status code should be 200")
```

```
def step_status(context):
    assert context.response.status_code == 200
```

5.12 Hooks, Scenario Outlines and Tags

```
# features/environment.py
def before_scenario(context, scenario):
    context.session = requests.Session()

def after_scenario(context, scenario):
    context.session.close()
```

Scenario outlines support parameterization through Examples tables. Tags such as `@smoke`, `@regression`, and `@api` allow selective execution.

```
behave --tags=smoke
behave --tags=api
behave -f allure_behave.formatter:AllureFormatter -o allure-results
```

5.13 Allure Reporting

```
allure serve allure-results
```

Allure results provide a visual summary of scenarios, steps, statuses, failures, and execution details. The report should be generated after the test execution completes.

6. Output with Clearly Labelled Screenshots

The following labelled screenshots are illustrative documentation samples representing expected execution views. They are not verified live execution captures from a specific API server.

Figure 1: REST vs SOAP Concepts

Figure 1: REST vs SOAP Concepts

REST: resource-oriented, commonly JSON, stateless HTTP operations

SOAP: protocol-oriented, XML envelope, formal standards

Selected approach for this experiment: Python requests + REST APIs

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 2: GET Request and JSON Response

Figure 2: GET Request and JSON Response

GET /users/1

HTTP 200 OK

JSON response: {"id": 1, "name": "Example User"}

Response converted using response.json()

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 3: Response Validation

Figure 3: Response Validation

Status code assertion: PASSED

Content-Type header: application/json

Required JSON key validation: PASSED

Response time and error checks recorded

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 4: POST Request with Payload and Headers

Figure 4: POST Request with Payload and Headers

POST /users
Headers: Content-Type: application/json
Payload: {"name":"Ahana","role":"QA Engineer"}
HTTP 201 Created
Returned resource identifier captured

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 5: Configuration, Authentication and Session

Figure 5: Configuration, Authentication and Session

Base URL loaded from config object
Bearer token passed through auth/header
requests.Session() reused connection and common headers
Cookies managed at session level

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 6: MySQL Data Driven API Test

Figure 6: MySQL Data Driven API Test

Database connection established
Test data fetched from example table
Rows converted into reusable request payloads
Connection closed after execution

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 7: PUT, PATCH and DELETE Operations

Figure 7: PUT, PATCH and DELETE Operations

PUT: complete resource replacement
PATCH: partial resource modification
DELETE: resource deletion
Status codes and response body validated

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 8: Advanced HTTP Response Handling

Figure 8: Advanced HTTP Response Handling

Text content read with `response.text`
Binary content read with `response.content`
Streaming handled with `stream=True`
Exceptions handled using `requests.exceptions`

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 9: Behave Step Definitions and Hooks

Figure 9: Behave Step Definitions and Hooks

Feature file uses Given / When / Then
`environment.py` contains `before_scenario` and `after_scenario`
Step definitions call reusable API client functions
Scenario context stores response data

Observation: The screenshot illustrates the expected output or framework state for this stage.

Figure 10: BDD Tags and Allure Reporting

Figure 10: BDD Tags and Allure Reporting

Tags: @smoke @regression @api
Selected scenarios executed by tag
Allure results generated from test run
Report displays passed, failed and skipped scenarios

Observation: The screenshot illustrates the expected output or framework state for this stage.

7. Result, Observation and Conclusion

The experiment demonstrates how Python can be used to automate REST APIs across the complete testing lifecycle. GET, POST, PUT, PATCH, and DELETE operations can be executed and validated through the requests library. Configuration objects, reusable payloads, authentication, sessions, cookies, and MySQL data improve maintainability and reusability.

Behave adds a business-readable BDD layer using feature files, step definitions, hooks, scenario outlines, and tags. Allure reporting provides execution visibility. The resulting framework supports modular, reusable, and data-driven API automation.